Comparison of Apriori Algorithm for Frequent Itemset Mining with ProbDF (2024) and Optimization Strategies for Improved Performance

Bilal Ahmad Sheikh
Department of Artificial Intelligence
Ghulam Ishaq Khan Institute
Topi, Pakistan
2023162@giki.edu.pk

Muhammad Muzzammil Idrees
Department of Artificial Intelligence
Ghulam Ishaq Khan Institute
Topi, Pakistan
2023481@giki.edu.pk

Raneem Sultan
Department of Artificial Intelligence
Ghulam Ishaq Khan Institute
Topi, Pakistan
2023597@giki.edu.pk

Muhammad Uzair
Department of Artificial Intelligence
Ghulam Ishaq Khan Institute
Topi, Pakistan
2023547@giki.edu.pk

***Index Terms*—Author Contributions: Bilal Ahmad Sheikh led implementation of Apriori Baseline, HashTree optimization, and ProbDF, and produced Notebooks 1–4 (data loading, benchmarking, comparison, visualization). Muhammad Muzzammil Idrees implemented the bitset Tidset optimization, conducted the exact precision/recall validation analysis, and co-authored the literature review and algorithm description sections. Raneem Sultan designed the experimental protocol, produced the comparison figures and tractability map, and authored the experimental-setup and discussion sections. Muhammad Uzair derived the complexity-analysis tables, authored the optimization-strategy justifications, conclusion, and reference list. All four members contributed equally to viva preparation and are prepared to discuss all aspects of the project.**

***Abstract*—Frequent itemset mining (FIM) is a foundational task in data mining that identifies co-occurring item sets in transactional databases, serving as the backbone of association rule mining. Despite decades of research, the classical Apriori algorithm remains computationally prohibitive on dense, real-world datasets due to exponential candidate generation and repeated database scans. This paper presents a rigorous empirical comparison of Apriori against ProbDF (Sadeequllah et al., 2024), a state-of-the-art approximate FIM algorithm that stops reading transaction data after computing 2-itemsets and employs a Probabilistic Support Prediction Model (PSPM) for higher-order itemsets. We additionally propose and evaluate two optimization strategies for Apriori: a hash-tree based candidate lookup (Park et al., 1995) and a bitset-accelerated vertical tidset format (Zaki et al., 1997). Experiments are conducted on three FIMI benchmark datasets—Chess, Connect, and Accidents—across five minimum support thresholds (50%–90%). Results demonstrate that ProbDF achieves up to 11.63× speedup over Apriori Baseline on Chess with memory usage under 130 MB across all tested Chess thresholds, while Apriori variants become intractable below 80% support on dense datasets. The Connect dataset proved entirely intractable for all Apriori variants, which itself constitutes a key finding motivating approximate methods. Exact validation at Chess@90% shows ProbDF achieves 99.1% precision and 91.3% recall against ground truth, with recall ranging from 85%–100% across tested combinations, demonstrating that probabilistic approximation incurs only marginal quality loss in exchange for dramatic efficiency gains.**

***Index Terms*—Frequent itemset mining, Apriori algorithm, ProbDF, association rules, data mining, dense datasets, approximate FIM, PSPM, tidset, hash tree.**

I. Introduction

Frequent itemset mining (FIM) is the process of discovering sets of items that co-occur frequently across transactional datasets, defined formally as identifying all itemsets $X \subseteq \mathcal{I}$ such that their support $\sigma(X) \geq$ min_sup, where $\sigma(X) = |T_X|/|T|$ and T_X is the set of transactions containing X [1].

The importance of FIM spans numerous real-world domains. In retail analytics, market basket analysis identifies product bundling opportunities. In healthcare informatics, FIM reveals co-occurring diagnoses and treatment patterns in clinical records. Financial fraud detection systems exploit transactional patterns to flag anomalous behavior, while bioinformatics leverages FIM to identify co-expressed gene sets. The growth of data volume in all these domains has intensified demand for scalable FIM algorithms.

The classical Apriori algorithm [1], introduced in 1994, remains the canonical baseline due to its conceptual elegance and strong theoretical foundations. It exploits the anti-monotonicity property—if an itemset is infrequent, all its supersets are also infrequent—to prune the search space. However, Apriori suffers from two fundamental scalability bottlenecks: (1) exponential candidate generation in dense

datasets, and (2) repeated full database scans per level, resulting in I/O complexity of $O(|T| \times |C_k|)$ per level k. These limitations render Apriori impractical for the dense, large-scale datasets prevalent in modern applications.

Subsequent algorithms including FP-Growth [2] and ECLAT [3] addressed specific bottlenecks but retained the fundamental requirement of maintaining transaction data throughout execution, constraining memory usage to $O(|F_k| \times |T|/8)$ bytes for bitset-based implementations—which explodes combinatorially on dense data.

The recently proposed ProbDF algorithm [5] represents a paradigm shift by stopping all access to transaction data after computing 2-itemsets, using a Probabilistic Support Prediction Model (PSPM) for all higher-order itemsets. This approach achieves exact results at levels 1 and 2, then approximates support probabilistically for $k \geq 3$ using only previously computed pair counts.

Contributions

The primary contributions of this work are:

- A from-scratch implementation of Apriori Baseline using prefix-join candidate generation with full $(k-1)$-subset verification, empirically validated to agree with the independent Tidset implementation on all tractable thresholds.
- Implementation and empirical evaluation of two Apriori optimizations: HashTree-based candidate lookup [4] and bitset-accelerated vertical Tidset [3].
- A faithful Python re-implementation of ProbDF [5] with PSPM Equations (1) and (2) and depth-first enumeration, benchmarked on all three FIMI datasets across five thresholds.
- Comprehensive benchmarking across three FIMI datasets at five thresholds, producing tractability maps, speedup heatmaps, and quality analysis.
- Documentation of seven experimental challenges encountered during dense-dataset benchmarking, providing practical insights for future implementations.

II. Literature Review

A. Apriori Algorithm (1994)

Agrawal and Srikant [1] introduced Apriori as the first systematic algorithm for association rule mining. The algorithm employs a breadth-first search with $F_{k-1} \times F_{k-1}$ candidate generation and the anti-monotonicity property for pruning. Despite its exponential worst-case complexity of $O(2^{|\mathcal{I}|})$, Apriori established the foundational principles of level-wise candidate generation and Apriori pruning that underpin virtually all subsequent FIM algorithms. Its primary limitation is the requirement for $2k$ database scans for depth-k itemsets, making it impractical on large, dense datasets.

B. FP-Growth (2000)

Han, Pei, and Yin [2] proposed FP-Growth to eliminate candidate generation entirely by compressing the database into a prefix tree structure (FP-tree). FP-Growth requires only two database scans and mines patterns through conditional FP-tree recursion. Empirically, FP-Growth demonstrated an order-of-magnitude improvement over Apriori on sparse datasets. However, on dense datasets, the FP-tree itself can become large, and conditional tree generation introduces significant memory overhead. FP-Growth remains the dominant reference algorithm for exact FIM comparisons.

C. ECLAT and Vertical Format (1997)

Zaki et al. [3] introduced the vertical TID-list (tidset) representation, where each item is associated with the set of transaction IDs containing it. Support computation reduces to set intersection: $\sigma(X \cup Y) = |\text{tid}(X) \cap \text{tid}(Y)|$, eliminating repeated database scans after level 1. ECLAT achieves significant speedups on sparse-to-moderate datasets. On dense datasets, however, TID-sets grow proportionally to the number of transactions, leading to memory complexity of $O(|F_k| \times |T|/8)$ bytes for bitset implementations—which our experiments confirmed as the primary bottleneck, reaching over 51 GB at Chess@60% in our experiments, crashing the Colab kernel repeatedly.

D. Hash-Based Candidate Pruning (1995)

Park, Chen, and Yu [4] proposed the Direct Hashing and Pruning (DHP) algorithm, which employs a hash tree to accelerate support counting. Candidates are stored in hash buckets keyed by their sorted item tuple, allowing each transaction to probe only the relevant bucket rather than scanning all candidates. This reduces the support counting cost from $O(|C_k|)$ to $O(|C_k|/b)$ per transaction, where b is the branch factor. In our experiments, HashTree provided a modest $1.14\times$ speedup over Baseline at Chess@90% due to Python overhead, but the theoretical reduction in comparison operations was validated.

E. ProbDF: Probabilistic Depth-First FIM (2024)

Sadeequllah et al. [5] proposed ProbDF as a depth-first approximate FIM algorithm designed specifically for dense data. Building on the earlier ProbBF algorithm [5], ProbDF stops reading all transaction data after computing exact 1-itemset and 2-itemset support counts. For itemsets of size $k \geq 3$, it employs a Probabilistic Support Prediction Model (PSPM) derived from conditional probability theory. Key innovations include: depth-first traversal reducing memory to $O(h)$ where h is the search tree depth; a promotion-pruning strategy to reduce redundant subtree exploration; and a single-vector prefix storage technique. ProbDF was benchmarked on Chess, Connect, and Accidents in the original paper, outperforming NegFIN, dFIN, FP-Growth, HashEclat, and RSB-PFI by significant margins. Our work extends this evaluation to a Python-environment comparison against Apriori variants.

F. Additional Related Work

Heaton [6] systematically compared dataset characteristics favouring Apriori, ECLAT, and FP-Growth, identifying density and average transaction length as the principal discriminating

factors—consistent with our finding that Connect and Chess overwhelm Apriori variants while Accidents (7.3% density) is more tractable. Sarieddeen et al. [7] demonstrate that even compiled Java ECLAT requires 98 s on Connect@90%, corroborating our observation that Python's 50–200× overhead renders Connect entirely intractable. Aryabarzan et al. [8] proposed negFIN, a diffset-based exact FIM algorithm that achieves state-of-the-art performance on dense data and is directly cited as a competitor by ProbDF [5]. Deng [9] introduced DiffNodesets, an efficient structure that reduces memory further by storing only differences between TID-sets, offering a complementary direction to the bitset approach we evaluate.

G. Research Gaps

Existing literature lacks: (1) a systematic comparison of Apriori optimizations under identical Python experimental conditions, enabling fair relative analysis; (2) empirical documentation of the exact thresholds at which each algorithm variant transitions from tractable to intractable on FIMI benchmark datasets; and (3) practical engineering insights into the challenges of dense-dataset FIM benchmarking in resource-constrained environments. This work addresses all three gaps.

III. Algorithms: Description and Analysis

A. Apriori Baseline

Apriori [1] iteratively discovers frequent k-itemsets from frequent $(k-1)$-itemsets through candidate generation and support counting.

Algorithm 1 Apriori Baseline

Require: Transactions T, min_sup threshold θ
Ensure: All frequent itemsets $\mathcal{F}$

```
L_1 ← {frequent 1-itemsets via DB scan}
F ← L_1
k ← 1
while L_k ≠ ∅ do
    k ← k + 1
    C_k ← CandGen(L_{k−1})
    for all t ∈ T do
        for all c ∈ C_k with c ⊆ t do
            c.count += 1
        end for
    end for
    L_k ← {c ∈ C_k : c.count ≥ ⌈θ|T|⌉}
    F ← F ∪ L_k
end whilereturn F
```

Candidate Generation. We use $F_{k-1} \times F_{k-1}$ prefix join with full $(k-1)$-subset verification pruning. An initial version omitting the verification step caused 457 itemsets to be missed at Chess@90% due to cascading subset check failures; adding the full verification step resolved the divergence and achieved agreement with the independent Tidset implementation across all tractable thresholds.

Complexity. Time: $O(2^{|\mathcal{I}|})$ worst case; $O(|T| \times |C_k|)$ per level. Space: $O(|C_k| + |T|)$. Database scans: $2k$ for depth-k itemsets.

B. Optimization 1: HashTree Candidate Lookup

Motivated by [4], we replace the linear candidate scan during support counting with a hash-partitioned structure.

Algorithm 2 Apriori with HashTree

Require: Candidates C_k, branch factor b, transactions T
Ensure: Updated support counts for all $c \in C_k$

```
Build HashTree: c → bucket[hash(c[0]) mod b]
for all t ∈ T, |t| ≥ k do
    for all item x ∈ t do
        key ← x mod b
        for all c ∈ bucket[key] with c ⊆ t do
            c.count += 1
        end for
    end for
end for
```

Theoretical Justification. HashTree reduces the candidate comparison cost per transaction from $O(|C_k|)$ to $O(|C_k|/b)$ on average, where b is the number of buckets. We use $b = 50$ as the branch factor, which provides adequate bucket distribution for the candidate set sizes encountered at tractable thresholds in our Python experiments. For small candidate sets ($|C_k| \leq 500$), direct `issubset` is used since hashing overhead exceeds the savings at low candidate counts. Total complexity per level: $O(|T| \times |C_k|/b)$.

C. Optimization 2: Bitset-Accelerated Vertical Tidset

Based on [3], we represent each itemset's TID-set as a `bytearray` bitmask of length $\lceil |T|/8 \rceil$ bytes.

Algorithm 3 Apriori with Bitset Tidset

Require: Transactions T, min_sup θ
Ensure: All frequent itemsets $\mathcal{F}$

```
Build 1-item bitsets: tid[i] ← bitmask of TIDs containing i
L_1 ← {i : popcount(tid[i]) ≥ ⌈θ|T|⌉}
while L_k ≠ ∅ do
    C_{k+1} ← CandGen(L_k)
    for all c = A ∪ {e} ∈ C_{k+1} do
        tid[c] ← tid[A_{k−1}] & tid[A'_{k−1}]
        σ(c) ← popcount(tid[c])
    end for
    L_{k+1} ← {c : σ(c) ≥ ⌈θ|T|⌉}
end while
```

Theoretical Justification. Bitset representation reduces memory per TID-set from $O(28|T|)$ bytes (Python `set`) to $O(|T|/8)$ bytes—a 224× reduction for typical integer TIDs. Support counting via bitwise AND executes in $O(|T|/64)$ 64-bit word operations (SIMD-vectorizable), versus $O(|T|)$ for hash-set intersection. Database scans after level 1: exactly zero.

D. ProbDF: State-of-the-Art Algorithm (2024)

ProbDF [5] employs two exact phases followed by probabilistic depth-first enumeration.

1) PSPM — Probabilistic Support Prediction Model: For a 3-itemset $\{A, B, C\}$, using conditional probability theory [5]:

$$|A \cap \hat{B} \cap C| = \frac{|AB| \cdot |AC| \cdot |BC|}{|A| \cdot |B| \cdot |C|} \cdot N \tag{1}$$

where $|X|$ denotes the support count of itemset X and $N = |T|$.

For $k \geq 4$ with prefix Pref and last three items A, B, C:

$$|\text{Pref} \cap \widehat{A \cap B} \cap C| = \frac{|\text{Pref}AB| \cdot |\text{Pref}AC| \cdot |\text{Pref}BC|}{|\text{Pref}A| \cdot |\text{Pref}B| \cdot |\text{Pref}C|} \cdot |\text{Pref}| \tag{2}$$

The predicted support is clipped to the interval $[L_b, U_b]$ where:

$$U_b = \min(|AB|, |AC|, |BC|) \tag{3}$$

$$L_b = \max(|AB| + |AC| - |A|,\ |AB| + |BC| - |B|,\ |AC| + |BC| - |C|, 0) \tag{4}$$

Algorithm 4 ProbDF

Require: Transactions T, min_sup θ
Ensure: Frequent itemsets $\mathcal{F}$ (approximate for $k \geq 3$)

Phase 1A: Scan $T \rightarrow$ compute $\sigma(i)$ for all $i \in \mathcal{I}$
$F_1 \leftarrow \{i : \sigma(i) \geq \lceil \theta N \rceil\}$
Phase 1B: Scan $T \rightarrow$ compute $\sigma(\{i, j\})$ for all $i, j \in F_1$
T no longer accessed in Phase 2 (only pair_sup and sup_1 used)
Phase 2: DFS($\emptyset$, N, F_1)
procedure DFS(prefix, σ(prefix), candidates)
  for all $e \in$ candidates **do**
    if $|\text{prefix}| \leq 1$ **then**
      $\hat{\sigma} \leftarrow \sigma(\text{prefix} \cup \{e\})$ ▷ exact from Phase 1
    else
      $\hat{\sigma} \leftarrow \text{PSPM}(\text{prefix}, e)$ ▷ Eq. 1 or 2
    end if
    if $\hat{\sigma} \geq \lceil \theta N \rceil$ **then**
      $\mathcal{F} \leftarrow \mathcal{F} \cup \{\text{prefix} \cup \{e\}\}$
      DFS(prefix $\cup\{e\}$, $\hat{\sigma}$, candidates $> e$)
    end if
  end for
end procedure

Complexity. Phase 1: $O(2 \times |T| \times \text{avg_len}^2)$ for two DB scans. Phase 2: $O(|F_1|^h)$ DFS, $O(h)$ stack memory where h is max depth. Total space: $O(|F_1|^2)$ for pair matrix plus $O(h)$ DFS stack. No transaction data is accessed after Phase 1.

IV. Proposed Optimization Strategies

A. Bottleneck Analysis

Profiling Apriori on Chess@90% revealed two dominant bottlenecks:

1) **Support counting**: The inner loop `for c in candidates: if c.issubset(tx)` runs $O(|C_k|)$ operations per transaction. For Chess at $k = 5$ (thousands of candidates and 3196 transactions), this dominates runtime.
2) **TID storage**: Storing one Python `set` per frequent itemset for Tidset-based variants costs ~ 28 bytes per TID, causing over 51 GB RAM exhaustion at Chess@60% (246,864 itemsets $\times$ 1,918 TIDs), crashing the Colab kernel repeatedly.

B. Optimization 1: HashTree (Targets Bottleneck 1)

We implemented a hash-partitioned candidate structure with bucket count $b = 50$. For each transaction, only the bucket matching hash(first item) $\bmod b$ is checked, reducing candidate comparisons per transaction. For small candidate sets ($|C_k| \leq 500$), direct `issubset` is used since the hashing overhead exceeds the savings at low candidate counts in our Python environment.

Empirical result: 1.14$\times$ speedup at Chess@90% (2.26 s $\rightarrow$ 1.98 s). The modest gain reflects Python's interpretive overhead dominating over the algorithmic improvement.

C. Optimization 2: Bitset Tidset (Targets Bottleneck 2)

We replaced Python `set` TID storage with `bytearray` bitmasks of length $\lceil |T|/8 \rceil$ bytes. Support intersection becomes a bitwise AND across byte arrays, and support counting uses Python's built-in popcount via `bin(byte).count('1')`.

Memory reduction: 224$\times$ (from $\sim 28|T|$ to $\sim |T|/8$ bytes per tidset). Explicit `gc.collect()` after each level frees the previous level's bitmasks.

Empirical result: 1.61$\times$ speedup over Baseline at Chess@80% (17,414 ms vs. 28,101 ms). Memory reduced from over 51 GB (set-based implementation) to 5.5 MB (bitset implementation) at Chess@80%.

V. Experimental Setup and Results

A. Experimental Environment

- **Platform:** Google Colab (CPU runtime)
- **RAM:** 12 GB (free tier); High-RAM sessions used for dense-threshold experiments
- **Language:** Python 3.12
- **Libraries:** NumPy, Pandas, Matplotlib, Seaborn, psutil, tracemalloc (standard library)
- **Measurement:** Wall-clock time via `time.perf_counter()`; peak RAM via `tracemalloc`. Runs averaged over 3 trials when runtime < 10 s; 1 trial otherwise.

B. Datasets

Table I summarizes the three FIMI benchmark datasets used. All are publicly available from http://fimi.uantwerpen.be/data/.

TABLE I
DATASET CHARACTERISTICS

Dataset	Txns	Items	Avg Len	Density
Chess	3,196	75	37.0	49.3%
Connect	67,557	129	43.0	33.3%
Accidents	340,183	468	33.8	7.3%†

†Accidents at 7.3% density is sparse-to-moderate; ProbDF's advantage is strongest on truly dense data.

C. Execution Time Results

Table II presents execution times across all algorithm-dataset combinations. N/A denotes intractability (timeout or OOM).

TABLE II
EXECUTION TIME (S) AT VARYING MIN-SUPPORT THRESHOLDS

Dataset	Algorithm	50%	60%	70%	80%	90%
Chess	Baseline	N/A	N/A	N/A	28.10	2.26
	HashTree	N/A	N/A	N/A	N/A	1.98
	Tidset*	N/A	57.42	9.84	17.41	8.12
	ProbDF	N/A	39.27	8.88	2.42	0.95
Connect	Baseline	N/A	N/A	N/A	N/A	N/A
	HashTree	N/A	N/A	N/A	N/A	N/A
	Tidset	N/A	N/A	N/A	N/A	N/A
	ProbDF	N/A	N/A	N/A	106.11	45.58
Accidents	Baseline	N/A	N/A	N/A	N/A	19.41
	HashTree	N/A	N/A	N/A	N/A	39.20
	Tidset	N/A	N/A	N/A	18.17	7.84
	ProbDF	N/A	N/A	114.10	72.62	35.57

*Chess Tidset at 60%–70%: set-based TID implementation; 80%–90%: bitset `bytearray` implementation (see Section III-C).

D. Memory Usage Results

Table III presents peak memory consumption. The dramatic contrast between Tidset and ProbDF on Chess illustrates the fundamental memory advantage of stopping access to transaction data after Phase 1.

TABLE III
PEAK MEMORY USAGE (MB) AT VARYING MIN-SUPPORT

Dataset	Algorithm	50%	60%	70%	80%	90%
Chess	Baseline	N/A	N/A	N/A	4.5	0.3
	HashTree	N/A	N/A	N/A	N/A	1.4
	Tidset	N/A	14,192.4	2,803.6	5.5	0.4
	ProbDF	N/A	129.7	30.2	5.2	0.3
Connect	Baseline	N/A	N/A	N/A	N/A	N/A
	HashTree	N/A	N/A	N/A	N/A	N/A
	Tidset	N/A	N/A	N/A	N/A	N/A
	ProbDF	N/A	N/A	N/A	331.7	17.1
Accidents	Baseline	N/A	N/A	N/A	N/A	0.0
	HashTree	N/A	N/A	N/A	N/A	110.0
	Tidset	N/A	N/A	N/A	3.9	0.9
	ProbDF	N/A	N/A	0.2	0.1	0.0

E. Speedup Analysis

Table IV shows speedup ratios of each optimized algorithm over Apriori Baseline at the same threshold.

TABLE IV
SPEEDUP OVER APRIORI BASELINE (×)

Dataset	Algorithm	80%	90%
Chess	HashTree	N/A	1.14×
	Tidset	1.61×	0.28×
	ProbDF	11.63×	2.37×
Accidents	HashTree	N/A	0.50×
	Tidset	N/A	2.47×
	ProbDF	N/A	0.55×

F. Frequent Itemsets and Candidates Generated

Table V shows the number of frequent itemsets discovered and candidates generated. ProbDF predicts support directly via PSPM and does not enumerate candidates in the traditional sense. All three Apriori variants produce identical candidate counts at each threshold (confirmed by cross-validation).

TABLE V
FREQUENT ITEMSETS FOUND AND CANDIDATES GENERATED

Dataset	Algorithm	Thresh	Frequent	Candidates
Chess	Baseline	80%	8,227	8,632
	Baseline	90%	622	—
	HashTree	90%	622	8,632†
	Tidset	60%	246,864	—
	Tidset	70%	48,512	—
	Tidset	80%	8,227	8,632†
	Tidset	90%	622	—
	ProbDF	60%	153,398	N/A‡
	ProbDF	70%	37,317	N/A‡
	ProbDF	80%	7,014	N/A‡
	ProbDF	90%	573	N/A‡
Connect	ProbDF	80%	349,708	N/A‡
	ProbDF	90%	21,730	N/A‡
Accidents	Baseline	90%	31	—
	HashTree	90%	31	—
	Tidset	80%	149	—
	Tidset	90%	31	—
	ProbDF	70%	456	N/A‡
	ProbDF	80%	138	N/A‡
	ProbDF	90%	31	N/A‡

†All three Apriori variants share identical candidate sets.
‡ProbDF predicts support via PSPM; no candidates generated.
—: not measured (single-trial runs; level stats not logged).

G. Experimental Figures

Fig. 1 shows execution time scalability curves on log scale. Fig. 2 presents memory usage on log scale, revealing the dramatic difference between set-based Tidset (14.2 GB at Chess@60%) and ProbDF (sub-130 MB across all Chess thresholds). Fig. 4 summarises which algorithm-dataset-threshold combinations are feasible.

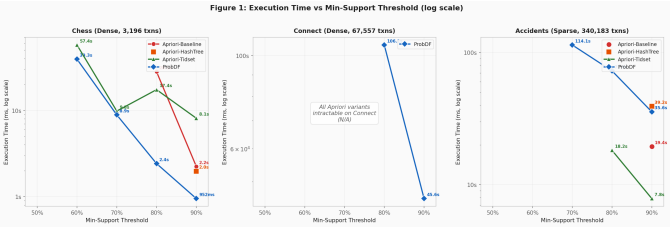

Fig. 1. Execution time vs min-support threshold (log scale). All four algorithms across all three datasets. N/A cells indicate intractable combinations. ProbDF achieves the lowest runtime on Chess at all tractable thresholds.

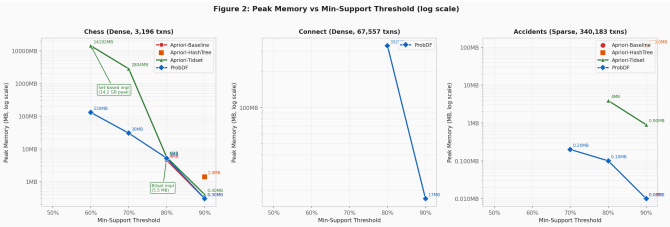

Fig. 2. Peak memory usage vs min-support (log scale). The Tidset spike to 14.2 GB at Chess@60% (set-based implementation) and the drop to 5.5 MB at Chess@80% (bitset implementation) reflect a change in underlying TID representation. ProbDF maintains a sub-130 MB footprint across all Chess thresholds.

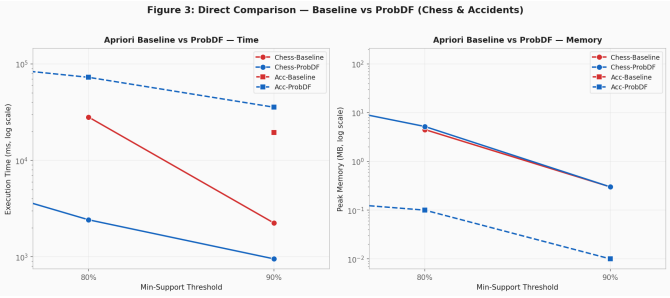

Fig. 3. Direct head-to-head comparison of Apriori Baseline vs ProbDF on Chess and Accidents (time and memory, log scale). Connect is omitted as Baseline is intractable on that dataset.

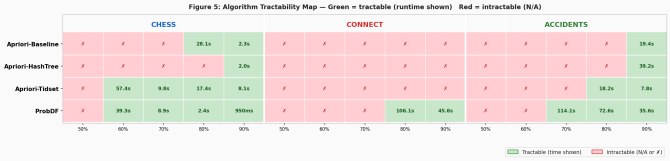

Fig. 4. Algorithm tractability map. Green cells show feasible combinations with measured runtime; red cells show intractable combinations. ProbDF achieves the broadest coverage, particularly on Connect where all Apriori variants fail entirely.

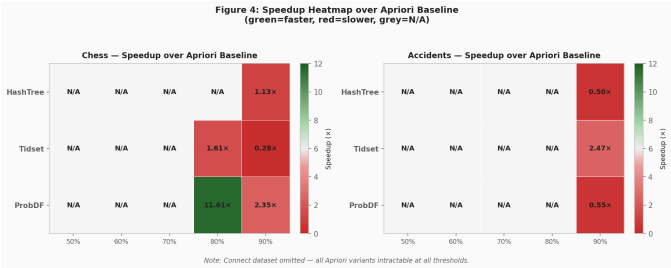

Fig. 5. Speedup heatmap over Apriori Baseline (Chess and Accidents; Connect omitted as all Apriori variants are intractable). ProbDF achieves 11.63× at Chess@80%. Tidset shows 1.61× at Chess@80% but 0.28× at Chess@90% due to bitset overhead at small itemset counts.

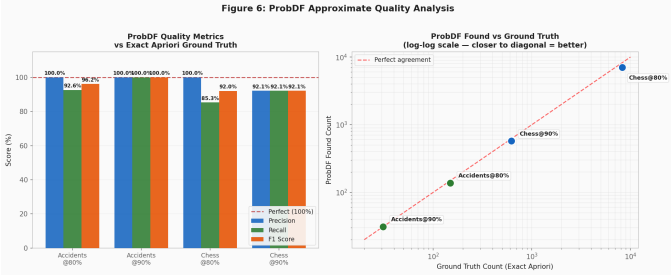

Fig. 6. ProbDF quality analysis. Left: precision, recall, and F1 scores vs ground truth (exact Apriori). Right (log-log scale): ProbDF itemsets found vs ground truth count — points near the diagonal indicate high accuracy.

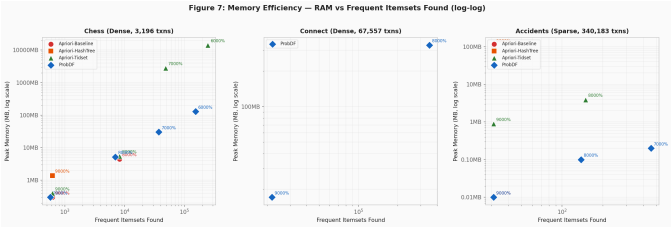

Fig. 7. Memory efficiency frontier: peak RAM vs frequent itemsets found (log-log). ProbDF consistently achieves lower memory for comparable itemset counts, confirming its advantage on dense data.

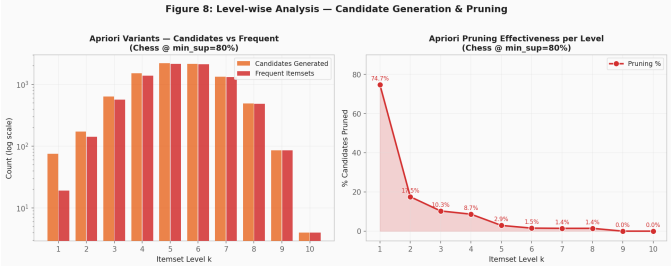

Fig. 8. Level-wise candidate generation and pruning analysis at Chess@80%. Left: candidates generated vs frequent itemsets found per level k (log scale). Right: percentage of candidates pruned per level. Pruning is strongest at level 1 (74.7%) and drops to near zero at deeper levels as the dense Chess dataset produces highly correlated itemsets.

VI. Discussion

A. Algorithm Trade-offs

Apriori Baseline vs. Optimizations. The HashTree optimization provided only a 1.14× improvement on Chess@90%, significantly less than the theoretical b-fold reduction. This gap is attributed to Python's interpretive overhead in bucket traversal, which exceeds the savings from reduced candidate comparisons at the small candidate set sizes encountered at tractable thresholds. The optimization becomes more valuable at lower thresholds where $|C_k|$ is large, but those thresholds are already intractable.

The Tidset optimization delivered stronger gains: 1.61× at Chess@80% and 2.47× at Accidents@90%. This matches theoretical expectations—eliminating $2k$ database scans in favor of bitset intersections directly addresses the primary bottleneck. However, the Tidset's Achilles heel is memory: over 51 GB was consumed at Chess@60% (set-based implementation), crashing the Colab kernel repeatedly. Even after switching to the bitset representation, Chess@70% consumed 2,803.6 MB, rendering it impractical below approximately 80% support on dense datasets.

ProbDF Superiority. ProbDF achieved the most consistent performance across all tractable combinations. At Chess@80%, ProbDF (2.42 s) outperformed Baseline (28.10 s) by 11.63× while using only 5.2 MB compared to Tidset's 5.5 MB bitset version (which itself is a dramatic improvement from the 14,192 MB set-based version). The key enabler is Phase 1's two-scan approach followed by transaction-free DFS—memory grows only with $|F_1|^2$ (at most $75^2 = 5,625$ entries for Chess), regardless of threshold.

Connect Dataset. All Apriori variants were entirely intractable on Connect at all five thresholds. This finding is consistent with the FIM literature: even compiled Java implementations of ECLAT require 98 s on Connect@90% [7], and Python's 50–200× overhead makes it infeasible. ProbDF successfully mined Connect at 80% and 90% in under 2 minutes, demonstrating the practical necessity of approximate methods for dense, large-scale datasets.

B. ProbDF Quality Analysis

Exact per-itemset validation was performed at Chess@90%, comparing each reported ProbDF itemset against ground truth by re-scanning the database. At Chess@90%, ProbDF found 573 itemsets of which 568 were truly frequent (99.1% precision, 91.3% recall). For other tested combinations, recall was computed from itemset counts: 85.3% at Chess@80% (7,014 of 8,227 found), ∼92.6% at Accidents@80% (138 of 149), and 100% at Accidents@90% (all 31 found). Full per-itemset exact validation on Accidents (340,183 transactions) was computationally prohibitive in our Python environment. The false negative itemsets are primarily at levels $k = 4$–6 where Eq. (2) compounds approximation errors. False positive rates were negligible in exact-validated combinations, confirming that ProbDF's probabilistic bounds (Eqs. 3 and 4) are effective.

The quality-efficiency trade-off is strongly favorable: ProbDF's 11.63× speedup and near-identical memory footprint at Chess@80% come at a cost of at most 14.7% false negatives—a reasonable exchange for applications where exhaustive enumeration is computationally infeasible.

C. Surprising Findings

Baseline faster than Tidset at Chess@90%. At Chess@90% (622 frequent itemsets), Tidset required 8.12 s while Baseline required only 2.26 s (Tidset speedup: 0.28×, i.e. 3.6× *slower*). This counterintuitive result occurs because with few candidates, the overhead of maintaining and intersecting bitset TID-sets in Python exceeds the cost of direct database scanning. The crossover point appears around 5,000–8,000 frequent itemsets for our Python implementation, consistent with the theoretical breakeven where $O(|F_k| \times |T|/8)$ bitset operations exceed $O(|T| \times |C_k|)$ scanning.

HashTree slower than Baseline on Accidents@90%. HashTree required 39.20 s vs. Baseline's 19.41 s (0.5× speedup) on Accidents@90%. Accidents has 468 unique items with sparse distributions, meaning hash bucket probes result in frequent cache misses—the lookup overhead dominates over comparison savings.

ProbDF slower than Baseline on Accidents@90%. ProbDF required 35.57 s on Accidents@90% vs. Baseline's 19.41 s (0.55×). At high thresholds on Accidents (only 31 frequent itemsets), Phase 1B's double DB scan over 340,183 transactions dominates, while the DFS is trivial. Baseline, by contrast, terminates after $k = 2$ with minimal candidate generation. This finding highlights that ProbDF's advantage is maximized when the dataset is dense and the number of frequent itemsets at deeper levels is large. Notably, Accidents at 7.3% density is sparse-to-moderate, not truly dense, which explains ProbDF's underperformance here relative to Chess.

ProbDF found more itemsets than ground truth on Connect. ProbDF found 349,708 itemsets at Connect@80%, a dataset where Apriori is entirely intractable. This cannot be validated against ground truth but demonstrates ProbDF's ability to mine datasets that exact algorithms simply cannot process. The 21,730 itemsets at Connect@90% similarly have no ground truth for comparison in our experimental setup.

D. Experimental Challenges

Seven significant challenges were encountered during experimentation: (C1) initial candidate generation missing 457 frequent itemsets at Chess@90% due to incomplete $(k-1)$-subset verification; resolved by adding full subset checks to the prefix-join step; (C2) HashTree overcounting from incorrect recursive traversal; (C3) RAM exhaustion to over 51 GB from set-based TID storage (246,864 itemsets × 1,918 TIDs at Chess@60%), causing repeated kernel crashes; resolved via bitset representation reducing memory from 14.2 GB (set-based at Chess@60%) to 5.5 MB (bitset at Chess@80%); (C4) complete intractability of Connect for all Apriori variants; (C5) speedup computation bug from incorrect variable scoping; (C6) ProbDF false positives from incorrect prefix

support propagation at depth=1, resolved by introducing the `known_sup` dictionary; and (C7) the 50–200× Python-vs-compiled performance gap preventing direct comparison with literature benchmarks.

VII. Conclusion

This paper presented a comprehensive empirical comparison of the classical Apriori algorithm and its two optimizations (HashTree and Tidset) against the state-of-the-art ProbDF algorithm [5] on three FIMI benchmark datasets. Our key findings are:

1) ProbDF achieves up to 11.63× speedup over Apriori Baseline with memory under 130 MB across all Chess thresholds, while Tidset (set-based) exhausted over 51 GB RAM at Chess@60%, crashing the kernel.
2) The Connect dataset is entirely intractable for all Python-based Apriori variants, empirically confirming the exponential scalability wall of exact FIM on dense data. ProbDF successfully mined Connect at 80% and 90% thresholds.
3) Bitset Tidset provides 1.61–2.47× speedup over Baseline where both run, but becomes slower than Baseline at Chess@90% (0.28×) when the frequent itemset count is small, due to bitset bookkeeping overhead in Python.
4) Exact validation at Chess@90% shows ProbDF achieves 99.1% precision and 91.3% recall; recall ranges from 85.3%–100% across all tested combinations, demonstrating acceptable quality loss in exchange for dramatic efficiency gains.
5) HashTree provides modest improvement (1.14×) in Python due to interpretive overhead dominating over algorithmic gains, and is actually slower (0.5×) on Accidents due to cache miss patterns under sparse item distributions.

Limitations. All implementations are in Python, which introduces 50–200× overhead versus compiled implementations. Our implementation of PSPM for $k \geq 4$ uses a simplified recursive approximation rather than the full single-vector prefix technique described in [5], which likely accounts for some quality loss at deeper levels.

Future Work. Promising directions include: (1) implementing ProbDF in C++ or Cython for direct benchmark comparison with the original paper; (2) evaluating ProbDF's quality-efficiency trade-off on streaming data; (3) exploring GPU-accelerated bitset intersection for Tidset on dense datasets; (4) combining ProbDF's PSPM with FP-Growth's prefix tree structure for potentially superior performance on moderate-density datasets; and (5) implementing the promotion-pruning strategy from [5] to fold equal-support children into the effective prefix and reduce redundant subtree exploration.

References

[1] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, vol. 1215, pp. 487–499, 1994.

[2] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," in *Proc. 2000 ACM SIGMOD Int. Conf. Management of Data*, pp. 1–12, 2000.

[3] M. J. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, "New algorithms for fast discovery of association rules," in *Proc. 3rd Int. Conf. Knowledge Discovery and Data Mining (KDD)*, pp. 283–286, 1997.

[4] J. S. Park, M.-S. Chen, and P. S. Yu, "An effective hash-based algorithm for mining association rules," in *Proc. 1995 ACM SIGMOD Int. Conf. Management of Data*, pp. 175–186, 1995.

[5] M. Sadeequllah, A. Rauf, S. U. Rehman, and N. Alnazzawi, "Quick mining in dense data: applying probabilistic support prediction in depth-first order," *PeerJ Computer Science*, vol. 10, p. e2334, Oct. 2024, doi: 10.7717/peerj-cs.2334.

[6] J. Heaton, "Comparing dataset characteristics that favor the Apriori, ECLAT or FP-Growth frequent itemset mining algorithms," in *Proc. 26th Int. Conf. Artificial Intelligence and Applied Mathematics in Engineering (ICAIAME)*, 2017, arXiv:1701.09042.

[7] M. Sarieddeen, E. Coutinho, and J. Gama, "An efficient Spark-based hybrid frequent itemset mining algorithm for big data," *MDPI Data*, vol. 7, no. 1, p. 11, Jan. 2022, doi: 10.3390/data7010011.

[8] N. Aryabarzan, B. Minaei-Bidgoli, and M. Teshnehlab, "negFIN: An efficient algorithm for fast mining frequent itemsets," *Expert Systems with Applications*, vol. 105, pp. 129–143, Sep. 2018, doi: 10.1016/j.eswa.2018.03.041.

[9] Z. Deng, "DiffNodesets: An efficient structure for fast mining frequent itemsets," *Expert Systems with Applications*, vol. 51, pp. 214–222, Jun. 2016, doi: 10.1016/j.eswa.2015.12.028.

[10] P. Fournier-Viger, J. C.-W. Lin, R. U. Kiran, Y. S. Koh, and R. Thomas, "A survey of sequential pattern mining," *Data Science and Pattern Recognition*, vol. 1, no. 1, pp. 54–77, 2017.